

PROJET B2 — MAI 2026

Créez votre propre Chatbot avec Claude AI

Python · API Anthropic · Conversation intelligente

Guide étape par étape pour construire un chatbot conversationnel en Python avec gestion de l'historique, personnalisation du comportement et streaming des réponses en temps réel.

Introduction

Qu'est-ce qu'un LLM ?

Un **Large Language Model (LLM)** est un modèle d'intelligence artificielle entraîné sur d'immenses quantités de texte. Il est capable de comprendre et de générer du langage naturel de manière cohérente et contextuelle.



Prompt

L'instruction ou la question que vous envoyez au modèle. La qualité du prompt influence directement la qualité de la réponse.



Conversation

Un échange multi-tours où l'historique des messages est renvoyé à chaque requête pour maintenir le contexte.



System Prompt

Une instruction cachée qui définit le comportement, le ton et le rôle du chatbot avant toute interaction.

Comment ça marche ?



Étape 1

Installation et configuration

1 Préparer l'environnement

Prérequis

- Python 3.9 ou supérieur installé
- Un compte Anthropic (console.anthropic.com)
- Une clé API (gratuite pour commencer)

Installer le SDK Anthropic :

```
# Installer le SDK
pip install anthropic

# Définir la clé API
export ANTHROPIC_API_KEY="sk-ant-votre-clé"
```

Obtenir une clé API :



Créer un compte sur console.anthropic.com



Aller dans Settings → API Keys



Cliquer 'Create Key' et copier la clé



Stocker la clé en sécurité (jamais dans le code public !)

■ Astuce

Les nouveaux comptes reçoivent des crédits gratuits.
Vérifiez votre numéro de téléphone pour les activer.

Étape 2

Premier appel à l'API

2

Envoyer un prompt et recevoir une réponse

Le principe est simple : on envoie un message (le **prompt**) à l'API et on reçoit une réponse générée par le modèle. Voici le code minimal :

```
import anthropic

client = anthropic.Anthropic()

message = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=[
        {"role": "user",
         "content": "Bonjour ! Explique
                    -moi ce qu'est un LLM."}
    ]
)

print(message.content[0].text)
```

Résultat attendu :

Un LLM (Large Language Model) est un modèle d'IA entraîné sur d'immenses corpus de texte. Il comprend et génère du langage naturel. On interagit avec lui via des prompts textuels.

Concepts clés de cet appel :

model	Le modèle Claude à utiliser (Sonnet = bon rapport qualité/coût)
max_tokens	Limite de longueur de la réponse (1 token $\approx$ 0.75 mot)
messages	La liste des messages — c'est ici que se passe la magie

Étape 3

Gérer l'historique de conversation

3

Donner de la mémoire au chatbot

Un LLM n'a **aucune mémoire** entre les requêtes. Pour qu'il se 'souviene' du contexte, il faut renvoyer **tout l'historique** des messages à chaque appel. C'est le code client qui gère cet historique.

```
# L'historique est une simple liste
history = []

# À chaque message utilisateur :
history.append({"role": "user",
               "content": user_input})

# On envoie TOUT l'historique
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=history # <- tout l'historique !
)

# On ajoute la réponse à l'historique
history.append({"role": "assistant",
               "content": response.content[0].text})
```

Exemple de conversation avec mémoire :

■ Je m'appelle Alice, je suis en B2.

■ Bonjour Alice ! Comment puis-je t'aider ?

■ Tu te souviens de mon prénom ?

■ Bien sûr, tu es Alice, étudiante en B2 !

Étape 4

Personnaliser le chatbot

4

System prompt et personnalités

Le **system prompt** est une instruction envoyée au modèle avant la conversation. Il permet de définir le rôle, le ton et les contraintes du chatbot. C'est l'outil principal du **prompt engineering**.

```
response = client.messages.create(  
    model="claude-sonnet-4-20250514",  
    max_tokens=1024,  
    system="Tu es un pirate amical.  
    Tu réponds avec des expressions  
    pirates comme Arrr! et Mille  
    sabords!",  
    messages=history  
)
```

Personnalités pré-définies dans le projet :



Assistant

Pédagogue, clair et concis



Amusant, avec du vocabulaire pirate



Professeur

Socratique, guide par les questions



Coach

Motivant, enthousiaste, actionnable

■ Prompt Engineering

Soyez précis et explicite. Un bon prompt système donne le rôle, le ton, la langue et les contraintes que le modèle doit respecter.

Étape 5

Streaming et commandes

5 Réponses en temps réel et UX

Le **streaming** permet d'afficher la réponse mot par mot au lieu d'attendre qu'elle soit entièrement générée. L'expérience est plus naturelle et réactive.

```
# Streaming avec le SDK Anthropic
with client.messages.stream(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    messages=history
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

Commandes interactives du chatbot

Le chatbot inclut des commandes spéciales pour une meilleure UX :

Commande	Description
<code>/help</code>	Afficher l'aide
<code>/reset</code>	Réinitialiser la conversation
<code>/persona</code>	Choisir une personnalité
<code>/system</code>	Modifier le prompt système
<code>/history</code>	Voir l'historique
<code>/quit</code>	Quitter le chatbot

■ Sans streaming

L'utilisateur attend 3-5s
sans retour visuel...

■ Avec streaming

Les mots apparaissent en
temps réel, comme un humain

Récapitulatif

Ce que vous avez appris

1 Configurer l'environnement

SDK Anthropic + clé API

2 Envoyer un prompt

`client.messages.create()`

3 Gérer l'historique

Liste de messages renvoyée à chaque appel

4 Personnaliser

System prompt et prompt engineering

5 Streaming + UX

Réponses en temps réel + commandes

Pour aller plus loin...

- Ajouter une interface web avec Streamlit ou Flask
- Connecter des documents locaux (RAG)
- Comparer plusieurs modèles (Ollama, OpenAI, Anthropic)
- Ajouter des fonctions : résumé, traduction, Q&A